

Latihan Bab 8
Queue & Ticket Counter Simulation



Oleh:

Nyssa Leilani Calista 25091397062

Mata Kuliah :

Struktur Data

PROGRAM STUDI MANAJEMEN INFORMATIKA
FAKULTAS VOKASI
UNIVERSITAS NEGERI SURABAYA
2026

Jawaban Latihan Bab 8

Soal 1 Kompleksitas Waktu Worst Case

Berikut analisis kompleksitas waktu worst case untuk setiap operasi/metode dalam class Ticket Counter Simulation:

Metode	Kompleksitas	Penjelasan
<code>__init__</code>	$O(n)$	Loop sebanyak <code>numAgents</code> (n) untuk membuat agen
<code>run</code>	$O(n \times k)$	Loop <code>numMinutes</code> (n) $\times$ 3 handler, tiap handler $O(k)$ agen
<code>printResults</code>	$O(1)$	Hanya operasi aritmatika dan print, tidak ada loop
<code>_handleArrival</code>	$O(1)$	Cek probabilitas dan enqueue satu penumpang
<code>_handleBeginService</code>	$O(k)$	Loop seluruh agen ($k = \text{numAgents}$) untuk cari agen bebas
<code>_handleEndService</code>	$O(k)$	Loop seluruh agen untuk cek yang sudah selesai

$n = \text{numMinutes}$, $k = \text{numAgents}$. Kompleksitas keseluruhan `run()` adalah $O(n \times k)$.

Soal 2 Eksekusi Manual (Enqueue saja)

Nilai yang memenuhi $i \% 3 == 0$: **0, 3, 6, 9, 12, 15**

i	Operasi	Isi Queue (front $\rightarrow$ rear)
0	enqueue(0)	[0]
3	enqueue(3)	[0, 3]
6	enqueue(6)	[0, 3, 6]
9	enqueue(9)	[0, 3, 6, 9]
12	enqueue(12)	[0, 3, 6, 9, 12]
15	enqueue(15)	[0, 3, 6, 9, 12, 15]

Isi akhir queue: [0, 3, 6, 9, 12, 15]

Soal 3 – Eksekusi Manual (Enqueue & Dequeue)

Catatan: $\text{elif} \rightarrow i \% 4 == 0$ hanya jika $i \% 3 \neq 0$. $i=0$ dan $i=12$ memenuhi keduanya $\rightarrow$ hanya enqueue.

i	Kondisi	Operasi	Isi Queue (front $\rightarrow$ rear)
0	$0 \% 3 == 0 \checkmark$	enqueue(0)	[0]
1	–	(tidak ada)	[0]
2	–	(tidak ada)	[0]
3	$3 \% 3 == 0 \checkmark$	enqueue(3)	[0, 3]

4	$4\%4==0$ ✓ (elif)	dequeue() → 0	[3]
5	—	(tidak ada)	[3]
6	$6\%3==0$ ✓	enqueue(6)	[3, 6]
7	—	(tidak ada)	[3, 6]
8	$8\%4==0$ ✓ (elif)	dequeue() → 3	[6]
9	$9\%3==0$ ✓	enqueue(9)	[6, 9]
10	—	(tidak ada)	[6, 9]
11	—	(tidak ada)	[6, 9]
12	$12\%3==0$ ✓	enqueue(12)	[6, 9, 12]
13	—	(tidak ada)	[6, 9, 12]
14	—	(tidak ada)	[6, 9, 12]
15	$15\%3==0$ ✓	enqueue(15)	[6, 9, 12, 15]

Isi akhir queue: [6, 9, 12, 15]

Soal 4 – Implementasi Metode yang Tersisa

Tiga metode yang perlu diimplementasikan adalah `_handleArrival`, `_handleBeginService`, dan `_handleEndService`. Berikut implementasinya:

```
import random

def _handleArrival(self, curTime):
    # Rule 1: A passenger arrives based on arrival probability
```

```

        if random.random() <= self._arriveProb:
            passenger = Passenger(curTime)
            self._passengerQ.enqueue(passenger)
            self._numPassengers += 1

def _handleBeginService(self, curTime):
    # Rule 2: Free agents begin serving passengers in queue
    for i in range(len(self._theAgents)):
        if self._theAgents[i].isFree() and not self._passengerQ.isEmpty():
            nextPassenger = self._passengerQ.dequeue()
            waitTime = curTime - nextPassenger.arrivalTime()
            self._totalWaitTime += waitTime
            self._theAgents[i].startService(nextPassenger, curTime +
            self._serviceTime)

def _handleEndService(self, curTime):
    # Rule 3: Agents whose service time has expired become free
    for i in range(len(self._theAgents)):
        if self._theAgents[i].isFinished(curTime):
            self._theAgents[i].stopService()

```

Penjelasan:

- `_handleArrival`: Membangkitkan penumpang baru berdasarkan probabilitas kedatangan (`arriveProb = 1/betweenTime`). Jika random number $\leq$ `arriveProb`, penumpang baru di-enqueue.
- `_handleBeginService`: Mencari agen yang bebas (`isFree`), lalu mengambil penumpang dari antrian untuk dilayani. Waktu tunggu dihitung dan dicatat.
- `_handleEndService`: Mengecek setiap agen, jika sudah selesai melayani (`isFinished`), agen dikembalikan ke status bebas.

Soal 5 – Modifikasi ke Satuan Detik

Modifikasi utama: mengganti parameter numMinutes menjadi numSeconds, dan semua nilai waktu menggunakan detik (1 menit = 60 detik).

```
# Ubah satuan dari menit ke detik
# Ganti numMinutes -> numSeconds
# betweenTime dalam detik (mis: 120 = setiap 2 menit)
# serviceTime dalam detik (mis: 180 = 3 menit)

class TicketCounterSimulation:
    def __init__(self, numAgents, numSeconds, betweenTime, serviceTime):
        self._arriveProb = 1.0 / betweenTime
        self._serviceTime = serviceTime
        self._numSeconds = numSeconds
        self._passengerQ = Queue()
        self._theAgents = Array(numAgents)
        for i in range(numAgents):
            self._theAgents[i] = TicketAgent(i+1)
        self._totalWaitTime = 0
        self._numPassengers = 0

    def run(self):
        for curTime in range(self._numSeconds + 1):
            self._handleArrival(curTime)
            self._handleBeginService(curTime)
            self._handleEndService(curTime)
```

Tabel Hasil Simulasi dengan Satuan Detik (serviceTime dan betweenTime dalam detik):

Num Seconds	Num Agents	Avg Service (s)	Time Between (s)	Avg Wait (s)	Served	Remaining
6000	2	180	120	149.4	49	2
30000	2	180	120	234.6	240	0
60000	2	180	120	655.8	490	14
300000	2	180	120	945.0	2459	6
600000	2	180	120	1270.2	4930	18
6000	3	240	120	30.6	51	0
30000	3	240	120	30.0	240	0
60000	3	240	120	63.6	501	3

Tabel di atas dihasilkan dengan mengonversi parameter: 100 menit = 6000 detik, service 3 menit = 180 detik, between 2 menit = 120 detik. Hasil wait time pun dalam satuan detik.

Soal 6 Fungsi Membalik Urutan Queue

Fungsi reverseQueue membalik urutan item dalam queue menggunakan stack sebagai struktur data bantu. Hanya operasi Queue ADT (enqueue, dequeue, isEmpty) yang digunakan pada queue.

```
def reverseQueue(queue):
    """
    Membalik urutan item dalam queue.
    Menggunakan stack (list Python) sebagai struktur data bantu.
    Hanya menggunakan operasi Queue ADT: enqueue, dequeue, isEmpty.
```

```

"""

tempStack = []

# Langkah 1: Pindahkan semua item dari queue ke stack
while not queue.isEmpty():
    tempStack.append(queue.dequeue())

# Langkah 2: Pindahkan kembali dari stack ke queue
# (stack bersifat LIFO sehingga urutan menjadi terbalik)
while len(tempStack) > 0:
    queue.enqueue(tempStack.pop())

return queue

# Contoh penggunaan:
# q = Queue()
# for x in [1, 2, 3, 4, 5]:
#     q.enqueue(x)
# reverseQueue(q)
# # Hasil: [5, 4, 3, 2, 1] (front = 5)

```

Aspek	Nilai	Keterangan
Kompleksitas waktu	$O(n)$	Dua iterasi masing-masing n elemen
Kompleksitas ruang	$O(n)$	Stack bantu menyimpan n elemen
Operasi Queue dipakai	enqueue, dequeue, isEmpty	Sesuai syarat soal (hanya Queue ADT)

Analisis Kompleksitas:

- Waktu: $O(n)$ — dua iterasi masing-masing sebanyak n elemen.
- Ruang: $O(n)$ — stack tambahan menyimpan n elemen.
- Hanya operasi Queue ADT yang digunakan (enqueue, dequeue, isEmpty), sesuai syarat soal.